Informe de pruebas

Proyecto: **Cifrador con números de 40 cifras**

Integrantes: **Juan Pablo Beltrán, Santiago Alejandro Céspedes , Jhonatan David Valdés**

1. Resumen

Fecha	**14 de octubre de 2026**
Entorno	Python 3.12.3, Linux
Archivo probado	`cifrador_desde_txt.py`
Cómo reproducir	`python3 pruebas.py` (tarda ~10 s; no escribe archivos de resultados, solo usa carpetas temporales)
Resultado final	**85 / 85 pruebas correctas**

2. Resultados por área

#	Área	Pruebas	Qué se verificó	Resultado
P1	Generación	4	10.000 números por técnica: todos de 40 cifras, todos distintos, primos dentro de ±4σ de lo esperado	✓
P2	Uniformidad	4	Chi-cuadrado (nivel 1 %) sobre la primera y última cifra	✓
P3	Reproducibilidad	9	Misma semilla = misma secuencia; otra semilla = otra; parámetro `cifras` (2, 10, 20, 40, 60)	✓
P4	Primalidad	6	Contraste contra una criba de Eratóstenes; primos de Mersenne; números de Carmichael	✓
P5	Cifrar/descifrar	14	4 técnicas × 3 algoritmos × 6 mensajes; longitudes en el borde de bloque	✓
P6	Aleatorización	3	Mismo mensaje cifrado dos veces: textos distintos que descifran igual	✓
P7	Claves y alteraciones	9	Semilla incorrecta, algoritmo equivocado, mensaje alterado	✓
P8	Propiedades de las claves	11	p y q primos, $n = p \cdot q$, $e \cdot d \equiv 1 \pmod{\varphi}$, determinismo, cifrado de 1.000 números	✓
P9	Semilla	3	Conversión de números, frases y semilla vacía	✓
P10	Lectura del `.txt`	13	5 formatos válidos y 8 errores detectados	✓
P11	Extremo a extremo	8	Ejecución real del programa: cifrar → copiar bloque → descifrar; no se crean archivos	✓
P12	Rendimiento	1	Preparar sesión < 5 s	✓
	Total	**85**		**85 ✓ / 0 ✗**

3. Datos medidos

Generación de 10.000 números de 40 cifras. Primos esperados por teoría: 109,3 (desviación típica ≈ 10,5).

Técnica	Únicos	Primos	%	χ^2 primera cifra (< 20,09)	χ^2 última cifra (< 21,67)
mersenne	10.000	120	1,20 %	9,9	10,0
sha256	10.000	112	1,12 %	1,5	7,6
lcg	10.000	112	1,12 %	8,3	6,7
xorshift	10.000	105	1,05 %	14,2	8,9

Los cuatro generadores dan ~1,1 % de primos, coherente con la teoría, y no se detectó sesgo en las cifras. Esto es solo una comprobación básica de uniformidad: no demuestra calidad criptográfica.

Primalidad (Miller-Rabin).

- 0 discrepancias contra la criba de Eratóstenes en los 200.000 números de 0 a 199.999.
- Reconoce como primos $2^{61}-1$, $2^{89}-1$, $2^{107}-1$, $2^{127}-1$ y $2^{521}-1$.
- Rechaza como compuestos: 561, 1105, 1729, 2465, 2821, 6601, 8911, 41041, 825265, 321197185 (Carmichael), 3215031751 (pseudoprimo fuerte), el producto $(2^{127}-1)(2^{89}-1)$ y $(2^{61}-1)^2$.

Claves generadas. En las 4 técnicas, `p` y `q` son primos de 40 cifras y `n = p·q` tiene 80 cifras (264–265 bits). Con RSA caben 27 o 28 bytes de mensaje por bloque (según el tamaño exacto de `n`); con ElGamal, 11.

Rechazo de claves incorrectas y alteraciones.

Algoritmo	Semilla incorrecta rechazada	Mensaje con 1 dígito alterado detectado
RSA	200 / 200	4.984 / 4.984
ElGamal	200 / 200	4.880 / 4.880
XOR	200 / 200	4.930 / 4.930

(El número de alteraciones varía entre algoritmos porque las posiciones que caen en un separador `:` o `,` no se alteran.)

Rendimiento (mensaje de 5.000 caracteres, media de 5 repeticiones; depende de la máquina).

Algoritmo	Cifrar	Descifrar
RSA	≈ 3 ms	≈ 42 ms
ElGamal	≈ 52 ms	≈ 49 ms
XOR	≈ 1,5 ms	≈ 1,6 ms

Preparar una sesión completa (generar 10.000 números, buscar primos y construir las claves) tarda ≈ 0,2 s.

4. Hallazgo durante las pruebas y corrección

La primera ejecución dio **84/85**: en una prueba con 200 mensajes alterados, 1 de ElGamal **no fue detectado**.

- **Diagnóstico.** Con una muestra mayor (100.000 alteraciones) se confirmó que no era casualidad: en ElGamal, ≈ **6 de cada 100.000** mensajes alterados se aceptaban devolviendo texto basura. RSA y

ElGamal solo comprobaban un byte marcador por bloque, y un bloque corrupto puede pasar esa comprobación por azar. Con ElGamal el módulo es solo un poco mayor que 2^{128}, así que cerca del 4 % de los bloques basura empiezan con el byte esperado. El cifrado XOR no tenía el problema porque ya incluía una suma de comprobación sobre todo el mensaje.

- **Corrección.** La suma de comprobación pasó a la clase base `Cifrador`, de modo que los tres algoritmos verifican el mensaje completo (4 bytes de SHA-256). Probabilidad de falso aceptado: ≈ 2^{-32}.
- **Verificación.** Tras el cambio: 0 aceptados en 58.596 alteraciones (ElGamal), 59.773 (RSA) y 59.214 (XOR). La prueba P7 se amplió de 200 a 5.000 intentos por algoritmo, porque con 200 un fallo de esta frecuencia casi nunca se vería.

5. Qué no cubren las pruebas

- `cifrador_interactivo.py` (versión con menú) solo recibió una prueba manual rápida, no la batería.
- Solo se probó en Linux con Python 3.12.
- No se probaron mensajes de gran tamaño (megabytes).